

detector.py — teoria i opis modułu detekcji

1. Rola modułu w systemie Stop & Go

Moduł detector.py jest odpowiedzialny za odpowiedź na pytanie: gdzie na obrazie z kamery znajduje się pojazd poprzedzający? Przyjmuje klatkę w skali szarości z modułu camera.py i zwraca prostokąt otaczający pojazd — bbox (x, y, w, h) w pikselach. Ta jedna krotka czterech liczb jest jedyną informacją przekazywaną dalej do distance.py.

Moduł rozwiązuje dwa odrębne problemy: detekcję (czy i gdzie jest pojazd na danej klatce) oraz śledzenie (jak efektywnie utrzymywać pozycję już wykrytego pojazdu między kolejnymi detekcjami). Oba problemy wymagają różnych algorytmów i są łączone w jedną spójną strategię.

2. HOG — Histogram of Oriented Gradients

HOG (Histogram of Oriented Gradients) to deskryptor cech obrazu zaproponowany przez Dalalę i Triggisa (2005) i opisany szczegółowo w Kaehler & Bradski, Learning OpenCV 3. Kluczowa idea: zamiast analizować wartości jasności pikseli, analizujemy lokalne wzorce krawędzi. Samochód widziany od tyłu ma charakterystyczny układ krawędzi — linia dachu, boki karoserii, tylna szyba — który jest rozpoznawalny niezależnie od koloru pojazdu czy warunków oświetlenia.

2.1 Potok przetwarzania HOG

Obliczanie deskryptora HOG przebiega przez cztery etapy:



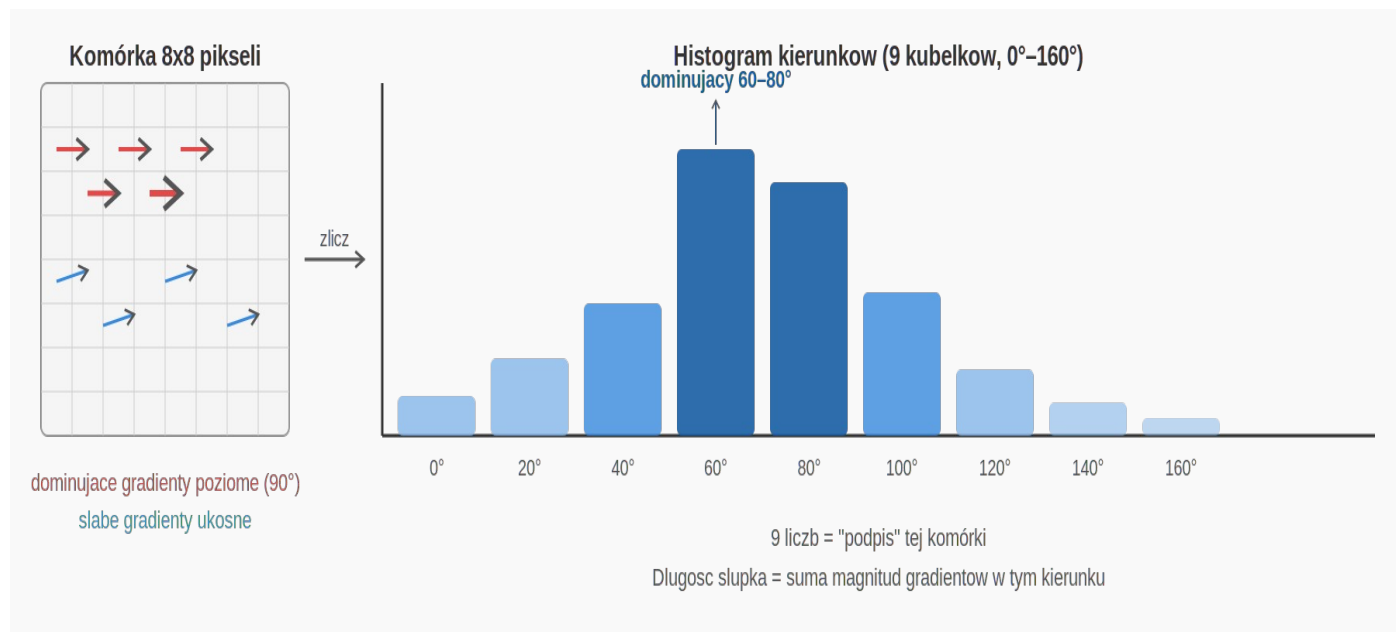
Rysunek 1. Potok obliczania deskryptora HOG — od pikseli do wektora cech

Etap 1 — Obliczenie gradientów. Dla każdego piksela obrazu wyznaczamy gradient poziomy Gx i pionowy Gy za pomocą prostych filtrów różniczkowych $[-1, 0, 1]$ oraz transponowany). Z gradientów wyznaczamy magnitude (siłę krawędzi) oraz kierunek (kąt krawędzi od 0° do 180°).

```
Gx = I(x+1,y) - I(x-1,y)    # gradient poziomy
Gy = I(x,y+1) - I(x,y-1)    # gradient pionowy
magnitude = sqrt(Gx2 + Gy2)
kierunek  = arctan(Gy / Gx)  # 0° - 180°
```

2.2 Komórki i histogramy

Etap 2 — Podział na komórki. Obraz jest dzielony na komórki 8×8 pikseli. Dla każdej komórki budowany jest histogram kierunków gradientów z 9 kubelkami ($0^\circ, 20^\circ, 40^\circ, \dots, 160^\circ$). Do każdego kubelka dodawana jest magnituda gradientu piksela o tym kierunku — dzięki temu krawędź silna (duży kontrast) waży więcej niż krawędź słaba (mały kontrast).



Rysunek 2. Komórka 8×8 z dominującymi gradientami poziomymi i odpowiadający jej histogram — dominuje kubełek $60\text{--}80^\circ$

Wynikiem jest 9 liczb opisujących tę komórkę — jej "podpis". Komórka na linii dachu samochodu będzie miała wysoki słupek przy 90° (krawędź pozioma). Komórka na bocznym słupku — wysoki słupek przy 0° lub 180° (krawędź pionowa). Ten wzorec jest stabilny przy różnych warunkach oświetlenia.

2.3 Normalizacja bloków

Etap 3 — Grupowanie w bloki. Każde cztery sąsiednie komórki (blok $2 \times 2 = 16 \times 16$ px) są grupowane razem i ich 36 liczb ($4 \text{ komórki} \times 9 \text{ kubelków}$) są normalizowane — dzielone przez normę L2 całego bloku. Normalizacja uniezależnia deskryptor od bezwzględnej jasności sceny: ten sam pojazd przy słonecznej i zachmurzonej pogodzie da taki sam znormalizowany wektor.

Dlaczego normalizacja bloków jest kluczowa?

Bez normalizacji: samochód w pełnym słońcu → gradienty ~ 150

 samochód w cieniu → gradienty ~ 30

SVM dostaje dwa zupełnie różne wektory dla tego samego obiektu.

Po normalizacji: oba przypadki → zbliżony wektor jednostkowy.

SVM rozpoznaje ten sam obiekt niezależnie od oświetlenia.

2.4 Klasyfikator SVM

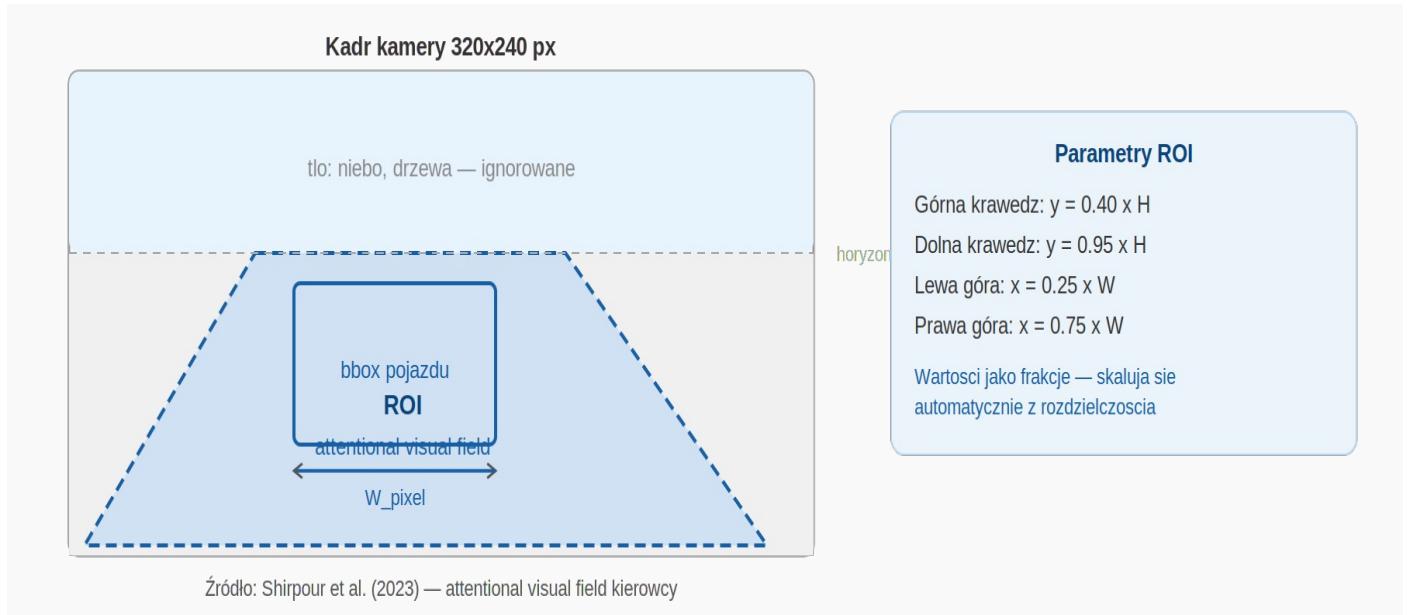
Etap 4 — Klasyfikacja. Złączony wektor HOG całego okna detekcji (64×128 px = 8×16 komórek = 7×15 bloków = 3780 liczb) jest podawany do klasyfikatora SVM (Support Vector Machine). SVM wyznacza hiperpłaszczyznę oddzielającą wektory pojazdów od wektorów tła w przestrzeni 3780 wymiarów.

OpenCV dostarcza gotowy, wytrenowany SVM przez `HOGDescriptor_getDefaultPeopleDetector()`. W projekcie używamy go jako punktu startowego. Docelowo można wytrenować własny SVM na zdjęciach konkretnego pojazdu RC — metoda `load_vehicle_svm()` jest przygotowana w klasie.

3. Region of Interest — attentional visual field

HOG skanuje obraz przesuwając okno detekcji krok po kroku. Przy klatce 320×240 i oknie 64×128 z krokiem 8px daje to kilka tysięcy pozycji do sprawdzenia — na Raspberry Pi to ~150 ms. Zbyt wolno dla systemu 30 fps.

Rozwiązaniem jest ograniczenie skanowania do obszaru zainteresowania (ROI). Pojazd poprzedzający prawie zawsze znajduje się w trapezie perspektywicznym przed pojazdem — pojęcie opisane przez Shirpour et al. (2023) jako attentional visual field kierowcy.



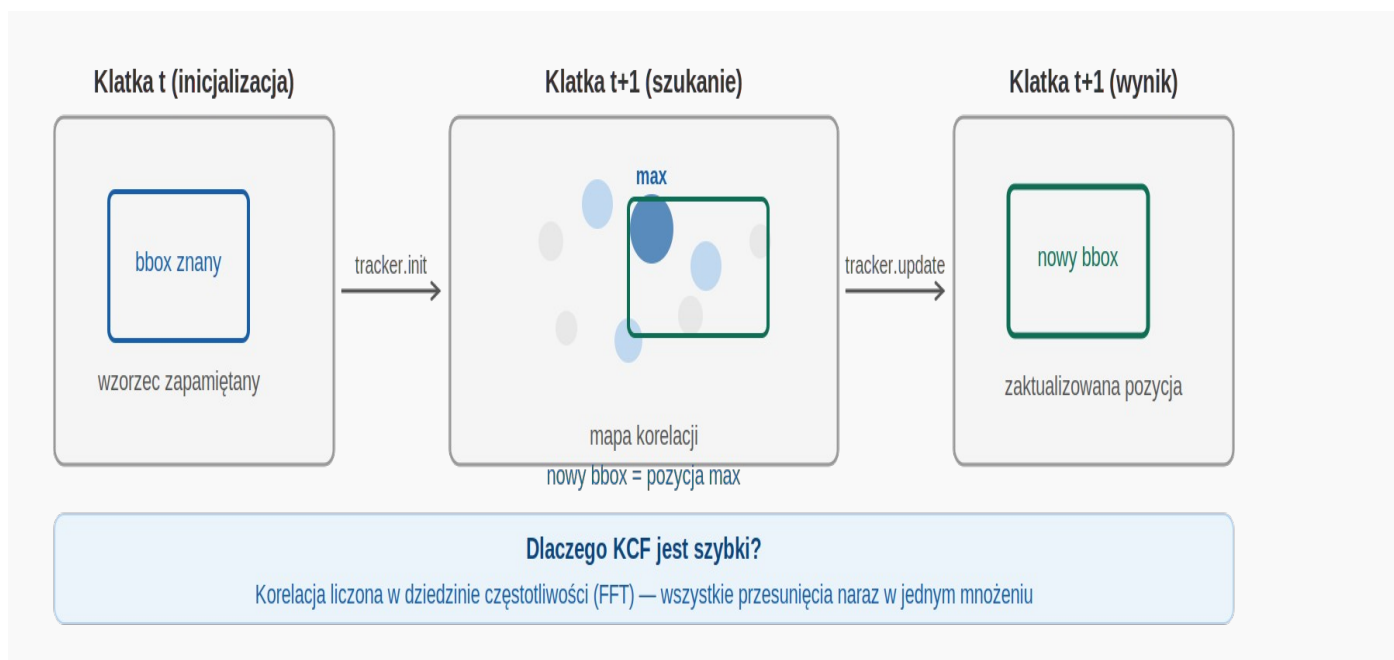
Rysunek 3. Trapezoidalny ROI na kadrze kamery. Pojazd poprzedzający zawsze mieści się w tym obszarze. Niebo i boki drogi są ignorowane.

Trapez jest definiowany jako frakcje szerokości i wysokości klatki — nie jako piksele. Dzięki temu ten sam kod działa przy 320×240 i 640×480 bez żadnych zmian. Frakcje opisują górną krawędź (okolice horyzontu, ~40% wysokości kadru) oraz dolną krawędź (przy pojeździe, ~95% wysokości kadru).

Maska ROI jest budowana raz w konstruktorze metodą fillPoly i przechowywana jako atrybut obiektu. W każdym obiegu pętli jedyną operacją to bitwise_and — logiczne AND każdego piksela z maską — co trwa mikrosekundy.

4. Śledzenie obiektu — KCF i CSRT

Pełna detekcja HOG na każdej klatce jest zbyt kosztowna. Rozwiązaniem jest strategia detect-then-track: HOG działa co N klatek, a między detekcjami obiekt jest śledzony szybkim trackerem który nie analizuje całego obrazu — tylko aktualizuje pozycję już znalezionego obiektu.



Rysunek 4. Zasada działania trackera KCF — wzorzec obiektu z klatki t jest szukany na klatce t+1 przez korelację. Pozycja maksimum mapy korelacji = nowa pozycja bbox.

4.1 KCF — Kernelized Correlation Filter

KCF (Kernelized Correlation Filter) to tracker oparty na filtrach korelacyjnych. Po inicjalizacji (przekazaniu klatki i bbox) KCF uczy się filtra opisującego wygląd obiektu. Na każdej kolejnej klatce przesuwa ten filtr po obszarze wokół poprzedniej pozycji i szuka miejsca o maksymalnej korelacji — to jest nowa pozycja pojazdu.

Kluczowa zaleta KCF: korelacja jest liczona w dziedzinie częstotliwości (transformata Fouriera). Zamiast liczyć podobieństwo dla każdej pozycji osobno, FFT pozwala obliczyć wszystkie przesunięcia naraz w jednym mnożeniu macierzowym. Stąd prędkość ~200 fps na typowym komputerze (~60 fps na Raspberry Pi 4).

Ograniczenia KCF — kiedy tracker traci obiekt:

- Okluzja: pojazd jest chwilowo zasłonięty (inny pojazd, słup)
- Skalowanie: pojazd zbliża się szybko — bbox zmienia rozmiar
- Obrót: pojazd skręca — wygląd się zmienia

Dlatego co REDETECT_EVERY_N klatek HOG weryfikuje i koryguje bbox.

4.2 CSRT — Channel and Spatial Reliability Tracker

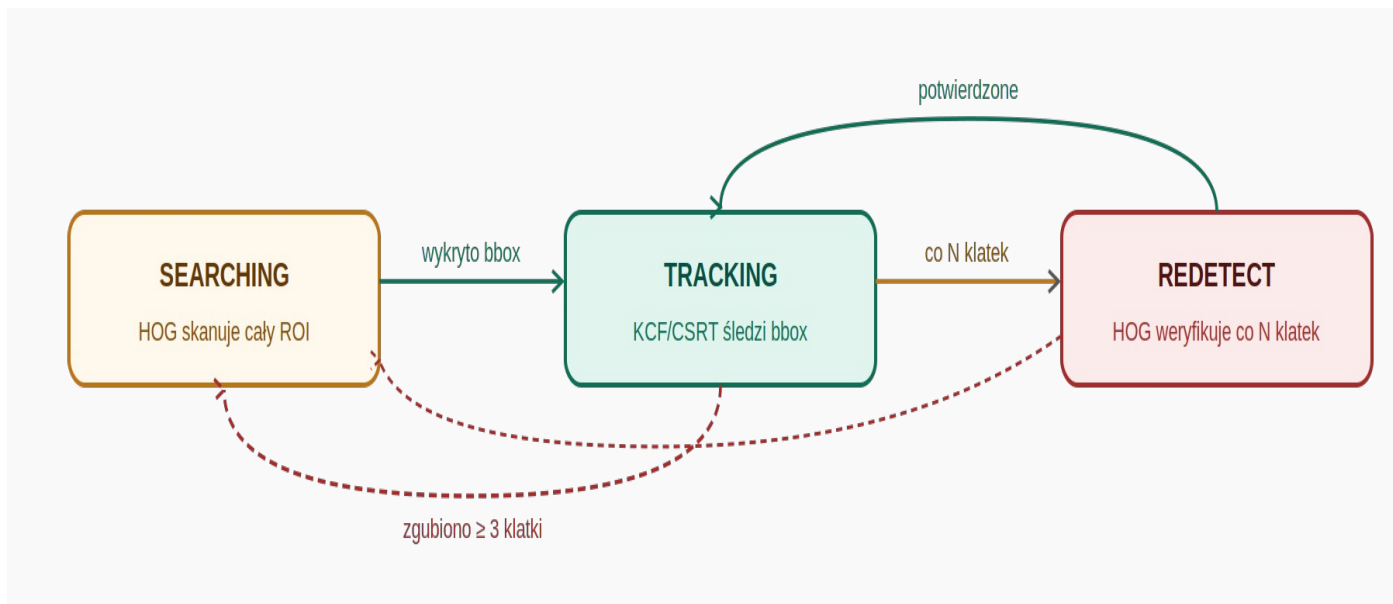
CSRT rozszerza KCF o maskę niezawodności przestrzennej (spatial reliability map). Tracker uczy się które piksele w bbox naprawdę należą do obiektu, a które to tło widoczne przez okna lub przy krawędziach bbox. Przy korelacji piksele tła są ważone niżej — tracker jest bardziej odporny na okluzję i zmiany wyglądu.

Cena: CSRT jest ~4-8× wolniejszy od KCF (~25 fps na typowym komputerze, ~15 fps na Raspberry Pi 4). Dla projektu zalecamy:

Raspberry Pi 4	KCF — szybszy, wystarczający przy typowych warunkach ruchu
Jetson Nano	CSRT — dokładniejszy, GPU akcelaruje operacje macierzowe
Środowisko testowe	KCF — szybsze iteracje przy strojeniu parametrów

5. Strategia detect-then-track i maszyna stanów

Detektor łączy HOG i tracker w jedną spójną strategię opartą na trzech stanach wewnętrznych:



Rysunek 5. Maszyna stanów modułu detector.py — przejścia między trybem szukania, śledzenia i redetekcji.

SEARCHING	HOG skanuje każdą klatkę w obszarze ROI. Gdy bbox zostanie znaleziony, tracker jest inicjalizowany i stan zmienia się na TRACKING.
TRACKING	KCF/CSRT aktualizuje bbox na każdej klatce. Co REDETECT_EVERY_N klatek HOG weryfikuje wynik. Gdy tracker nie daje wyniku przez 3 klatki z rzędu — powrót do SEARCHING.
REDETECT	HOG uruchamia się co N klatek podczas TRACKING. Jeśli znajdzie bbox — reinicjalizuje tracker (koryguje dryf). Jeśli nie znajdzie — tracker kontynuuje.

5.1 Histereza przy gubieniu obiektu

Parametr `_lost_count >= 3` zapobiega natychmiastowemu resetu do SEARCHING przy pojedynczym niepowodzeniu trackera. Jedna klatka bez wyniku może wynikać z okluzji, szumu, lub chwilowego wyjścia pojazdu poza ROI. Trzy kolejne klatki bez wyniku to wiarygodny sygnał że pojazd zniknął.

Dlaczego reinicjalizujemy tracker zamiast go aktualizować?

Ani KCF ani CSRT nie mają metody `reset()` — wywołanie `init()` na istniejącym obiekcie z nowym bbox daje niespójne wyniki wewnętrzne. Dlatego każda reinicjalizacja to `new TrackerKCF_create()` — nowy obiekt z czystym stanem, gotowy do nauki od nowa na nowym bbox.

6. Parametry detectMultiScale

Metoda `detectMultiScale` jest sercem detekcji HOG. Skanuje obraz w wielu skalach (piramida obrazu) i na każdym poziomie przesuwając okno detekcji. Trzy parametry mają największy wpływ na wydajność i dokładność:

winStride=(8,8)	Krok przesuwania okna w pikselach. Mniejszy = dokładniej ale wolniej. (8,8) to dobry kompromis dla prototypu. (16,16) jest ~4× szybszy przy zauważalnej utracie dokładności.
padding=(4,4)	Margines wokół okna skanowania. Pomaga gdy pojazd jest przy krawędzi ROI. Zwiększenie nie wpływa znacząco na czas.
scale=1.05	Współczynnik piramidy skalowania. 1.05 = drobne kroki, lepsza detekcja różnych rozmiarów, więcej poziomów. 1.10 jest ~2× szybszy, mniej precyzyjny w skali.

6.1 Filtrowanie wyników

`detectMultiScale` może zwrócić wiele nakładających się okien dla jednego pojazdu. Wewnętrznie OpenCV stosuje Non-Maximum Suppression (NMS) przez parametr `groupThreshold` (domyślnie 2) — okna ze zbyt małą liczbą "głosów" są odrzucane.

Pozostałe wyniki są filtrowane w kodzie przez trzy kryteria:

```
# Kryterium 1: minimalny rozmiar bbox
box[2] >= MIN_BBOX_WIDTH    # szerokość >= 20 px
box[3] >= MIN_BBOX_HEIGHT   # wysokość >= 15 px

# Kryterium 2: minimalna pewność SVM
confidence >= HOG_CONFIDENCE_THRESHOLD # 0.3

# Kryterium 3: wybór najlepszego
best = max(valid, key=lambda bw: bw[1]) # max confidence
```

7. Interfejs publiczny

<code>update(frame_gray)</code>	Główna metoda. Przyjmuje klatkę GRAY 320×240, zwraca bbox (x,y,w,h) lub None. Wywołać raz na każdą klatkę w pętli głównej.
<code>draw_debug(frame)</code>	Rysuje bbox, ROI i informacje stanu na kolorowej klatce. Tylko do testów — nie używać w pętli produkcyjnej.
<code>state</code>	Property (tylko odczyt). Zwraca 'SEARCHING' lub 'TRACKING'.
<code>bbox</code>	Property (tylko odczyt). Zwraca ostatni znany bbox lub None.
<code>load_vehicle_svm(p)</code>	Ładuje własny SVM wytrenowany na pojeździe RC. Zastępuje domyślny detektor pieszych.

8. Odniesienia literaturowe

1. Kaehler A., Bradski G. — Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library, O'Reilly Media, 2016.

Rozdziały 12–14: deskryptory HOG, piramidy obrazu, `detectMultiScale`.

Rozdział 15: klasyfikatory SVM, trening i użycie wbudowanych detektorów.

2. Shirpour M., Khairdoost N., Bauer M.A., Beauchemin S.S. — Traffic Object Detection and Recognition Based on the Attentional Visual Field of Drivers. IEEE Transactions on Intelligent Vehicles, vol. 8, no. 1, pp. 594–604, Jan. 2023.

Koncepcja attentional visual field — podstawa projektowania trapezoidalnego ROI.

3. Dalal N., Triggs B. — Histograms of Oriented Gradients for Human Detection. CVPR 2005.

Oryginalna publikacja opisująca algorytm HOG będący podstawą deskryptora w OpenCV.